- `ValueTask<T>`—It's not shown in listing 7.2, but many of the APIs in ASP.NET Core use `ValueTask` instead of the more common `Task` for APIs that normally complete asynchronously but *may* complete asynchronously. You can read about why they were introduced and when to use them at http://mng.bz/o1GM.

Don't worry if you're not familiar with these constructs. C# is a fast-moving language, so keeping up can be tricky, but there's generally no reason you need to use the new features. Nevertheless, it's useful to be able to recognize them so that you can read and understand code that uses them.

If you're keen to embrace new features, you might consider implementing the `IParsable` interface when you implement `TryParse`. This interface uses the `static abstract interfaces` feature, which was introduced in C# 11, and requires implementing both a `TryParse` and `Parse` method. You can read more about the `IParsable` interface in the announcement post at http://mng.bz/nW2K.

Now we've looked extensively at binding simple types to route values, query strings, and headers. In section 7.3 we'll learn about binding to the body of a request by deserializing JSON to complex types.

7.3 Binding complex types to the JSON body

Model binding in minimal APIs relies on certain conventions to simplify the code you need to write. One such convention, which you've already seen, is about binding to route parameters and query string values. Another important convention is that minimal API endpoints assume that requests will be sent using JSON.

Minimal APIs can bind the body of a request to a single complex type in your endpoint handler by deserializing the request from JSON. That means that if you have an endpoint such as the one in the following listing,